COLLEGE OF ENGINEERING, GUINDY

DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY

MINI PROJECT REPORT

SMART CV ANALYSER USING C

SUBJECT: DATA STRUCTURES USING C

SUBMITTED BY:

S. NO.	NAME	REGISTER NUMBER
1	YUTHIKA K	2025115031
2	KALAIVANI E	2025115027
3	DHARSHINI R	2025115063

1. INTRODUCTION

The Smart CV Analyser is a console-based mini project developed using the C programming language. It demonstrates how data structures can be used to organise candidate profiles and compare them with the requirements of a job role. The program accepts candidate details such as name, education, career objective, years of experience, project description, and skills. The user can then enter a target job role and its requirements. The program compares the entered criteria with the candidate profile, calculates a rule-based match score, identifies missing skills, and displays suggestions for improvement.

The project uses a singly linked list to store candidate profiles during execution, a queue to manage CVs waiting for analysis, and a stack to maintain analysis history. A menu-driven interface allows the user to perform operations without a graphical frontend. The program is designed to compile with GCC and does not require a backend, external library, or file handling.

2. OBJECTIVES

- To accept and store candidate profile details through user input.
- To display all candidate CV profiles entered during the current execution.
- To manage CVs waiting for analysis using a queue and FIFO order.
- To compare candidate skills with the skills required for a job role.
- To evaluate education, experience, and project information using simple rules.
- To calculate and display a match score out of 100.
- To suggest missing skills and profile details that may need improvement.
- To maintain recent analysis results using a stack and demonstrate LIFO order.
- To apply fundamental data structures and C programming concepts to a practical problem.

3. SOFTWARE REQUIREMENTS

Component	Requirement
Programming language	C
Compiler	GCC
Development environment	Visual Studio Code or another C editor
Execution	Command-line terminal
External libraries	Not required; standard C libraries are used

4. DATA STRUCTURES AND CONCEPTS USED

4.1 Structures (struct) and typedef

Structures group related information under one data type. The Candidate structure stores a candidate's ID, name, education, career objective, experience, project description, skills, and a pointer to the next candidate. Separate structures represent queue nodes and analysis-history records. The typedef keyword makes these structure types easier to use in declarations.

4.2 Arrays and String Handling

Character arrays store text such as names, education, job roles, project descriptions, and skills. A two-dimensional character array stores multiple skills for one candidate. Input is read with fgets() to allow spaces in text. String operations are used to remove the newline and compare skill names without case sensitivity.

4.3 Singly Linked List

The linked list stores candidate profiles. Each node contains one candidate's details and a pointer to the next node. The head pointer identifies the first candidate. When a new candidate is entered, memory is allocated dynamically and the node is appended to the end of the list. The findCandidate() function searches the list by candidate ID.

Illustration: candidate linked list

head	Candidate 1 ID, name, skills	Candidate 2 ID, name, skills	Candidate 3 ID, name, skills	**NULL**
→	next →	next →	next →	End

4.4 Queue

The queue stores candidate IDs that are waiting for analysis. It follows FIFO (First In, First Out): the first candidate added to the queue is the first one removed for analysis. The front pointer refers to the next candidate to be analysed, while rear refers to the last queued candidate. Enqueueing adds a node at the rear; analysing the next CV removes the node at the front.

Illustration: CV analysis queue

front	CV ID 1	CV ID 2	CV ID 3	**rear**

4. DATA STRUCTURES AND CONCEPTS USED (CONTINUED)

4.5 Stack

The stack stores analysis results and follows LIFO (Last In, First Out). Each new analysis result is pushed onto the top of the stack. When the history is displayed, the most recent analysis appears first. This provides a simple way to review results from the current program session.

Illustration: analysis history stack

TOP
Most recent analysis
Previous analysis
Older analysis

4.6 Dynamic Memory Allocation

The program uses malloc() to create candidate, queue, and history nodes while it runs. If allocation fails, the program displays an error instead of using an invalid pointer. The freeMemory() function releases the allocated nodes when the user exits, helping avoid memory leaks.

4.7 Searching and Rule-Based Matching

Candidate records are searched by ID using the linked list. During analysis, each required skill is compared with the candidate's skills. The matching routine is case-insensitive, so entries such as C and c are treated as the same skill. Education is checked against a user-provided keyword, experience is compared with the minimum requirement, and project presence is checked when the role requires a project.

4.8 Functions and Control Statements

Functions separate the program into manageable tasks. The main() function presents the menu and uses switch-case to call the selected operation. A do-while loop repeats the menu until Exit is selected. If statements validate input and choose outcomes, while loops traverse linked lists and process queue or stack nodes.

5. SYSTEM MODULES

5.1 Candidate Profile Entry

The addCandidate() function requests candidate details and stores them in a newly allocated linked-list node. Each candidate receives a unique ID during the current run. The number of skills is restricted to the supported range.

5.2 Candidate Display and Search

The displayCandidates() function lists all profiles. The findCandidate() function searches by ID and is used by queue operations and analysis to locate the selected profile.

5.3 Queue Management

The enqueueCV() function adds a selected candidate ID to the rear of the queue. The displayQueue() function lists candidates waiting for analysis. The analyseNextCV() function removes the oldest queue entry before collecting job requirements and performing the analysis.

5. SYSTEM MODULES (CONTINUED)

5.4 Job Requirement Entry

For each analysis, the user enters the target job role, an education keyword or ANY, minimum years of experience, required skills, and whether a project is required. This makes the criteria user-defined rather than fixed in the source code.

5.5 CV Analysis and Score Calculation

The program compares the candidate's skills with the required skills and awards up to 70 points for skill matching. Education, experience, and project criteria contribute 10 points each. The total score is therefore out of 100. The program labels scores of 80 or above as a strong match, scores from 60 to below 80 as a moderate match, and scores below 60 as a low match.

Scoring formulas

Component	Calculation	Maximum
Skill score	(Matched required skills ÷ Total required skills) × 70	70
Education	10 points if requirement matches	10
Experience	10 points if candidate meets minimum	10
Project	10 points if project criterion is satisfied	10
Total	**Sum of all components**	**100**

If no required skills are entered, the current implementation assigns a default skill score of 70. This is a design choice in the sample program and should be interpreted carefully. The score is a simple indicator based on entered criteria, not a scientifically validated hiring assessment.

5.6 Improvement Suggestions

The program prints each required skill that was not found in the candidate's profile. It also points out an education mismatch, insufficient experience, a missing project when relevant, or a missing career objective. These suggestions help the user understand which entered criteria were not met.

5.7 Analysis History

After an analysis, the program pushes the candidate ID, candidate name, job role, and score onto the history stack. The history module displays the latest result first.

6. ALGORITHM

- Step 1: Start the program and initialise the linked-list, queue, and stack pointers.
- Step 2: Display the main menu and accept the user's choice.
- Step 3: If choice 1 is selected, accept candidate details and append the new candidate node to the linked list.
- Step 4: If choice 2 is selected, traverse and display all candidate profiles.
- Step 5: If choice 3 is selected, validate the candidate ID and enqueue it for analysis.
- Step 6: If choice 4 is selected, traverse and display the pending analysis queue.
- Step 7: If choice 5 is selected, remove the front candidate from the queue and accept the job requirements.
- Step 8: Compare required skills, education, experience, and project criteria with the selected profile.
- Step 9: Calculate the score, display the match category and improvement suggestions, and push the result onto the history stack.

• Step 10: If choice 6 is selected, display analysis history from the top of the stack.

• Step 11: If choice 7 is selected, release allocated memory and terminate the program.

• Step 12: For any invalid choice, display an error message. Otherwise, repeat the menu until Exit is selected.

7. PROGRAM FLOW

The following flow diagram summarises the main control flow of the Smart CV Analyser.

START
Initialise linked list, queue and stack
Display menu and read choice
Select operation using switch-case
Add / Display CV / Queue / Analyse / History
Choice = Exit?
YES → Free memory → STOP
NO → Return to menu

8. SAMPLE MENU

The program displays the following menu when it starts:

```
1. Add Candidate CV
2. Display All CVs
3. Add CV to Analysis Queue
4. Display Analysis Queue
5. Analyse Next CV
6. Display Analysis History
7. Exit
```

9. SAMPLE INPUT AND OUTPUT

Example candidate details

```
Name: Arun
Education: B.Tech IT
Objective: Software developer
Experience: 0 years
Project: Student Management System
Skills: C, SQL, HTML
```

Example job criteria

```
Role: Software Developer
Education keyword: B.Tech
Minimum experience: 0 years
Required skills: C, SQL, Python
Project required: Yes
```

Illustrative analysis output

```
Candidate: Arun
Matched skills: 2 out of 3
Skill score: 46.67 / 70
Education: MATCH
Experience: MATCH
Project: MATCH
Overall score: 76.67 / 100
Result: MODERATE MATCH
```

The sample above is illustrative. The actual output depends on the information entered by the user. Since C and SQL match but Python does not, the program suggests developing the missing Python skill.

10. ADVANTAGES

- Accepts candidate and job information through interactive user input.
- Organises multiple candidate profiles using a linked list.
- Processes queued CVs in FIFO order.
- Displays the latest analysis result first using a stack.
- Automatically calculates a score and identifies missing required skills.
- Uses standard C libraries and can be compiled with GCC.
- Demonstrates several data structures in one menu-driven application.

11. LIMITATIONS

- Candidate profiles and analysis history exist only in memory and are lost when the program exits.
- The program does not read PDF or Word CV files; users must enter candidate details manually.
- Skill matching uses exact skill-name comparison and does not understand synonyms or equivalent skills.
- The score is based on simple rules and user-entered criteria; it is not an AI or machine-learning model.
- There is no graphical frontend, online database, or backend service.
- The candidate ID counter restarts when the program is run again.
- Education matching is based on a keyword, which may not capture every valid qualification.

12. FUTURE ENHANCEMENTS

- Add file handling or a database to save candidate profiles and analysis results permanently.
- Create a graphical or web interface if permitted by future project requirements.
- Improve matching using skill categories, synonyms, and weighted job-specific criteria.
- Add validation for blank names, duplicate skills, and more input edge cases.
- Allow comparisons between multiple candidates for the same job role.
- In a future advanced version, integrate a trained model or natural-language processing system, if the project scope and tools allow it.

13. CONCLUSION

The Smart CV Analyser demonstrates how fundamental data structures can be applied to a candidate-screening task using C. The linked list stores candidate profiles, the queue manages CVs waiting for analysis according to FIFO order, and the stack maintains analysis history according to LIFO order. The program accepts user-defined job requirements, compares them with candidate details, calculates a rule-based score, and identifies missing skills or profile information. Although it does not use machine learning or permanent storage, it provides a practical demonstration of structures, pointers, dynamic memory allocation, linked lists, queues, stacks, searching, string handling, and menu-driven programming.

14. REFERENCES

- Class notes and learning materials on Data Structures Using C.
- C programming references covering structures, pointers, linked lists, stacks, queues, and dynamic memory allocation.
- GCC documentation and C standard-library references for compilation and string handling.